HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN 1



BÁO CÁO MÔN HỌC
PHÁT TRIỂN CÁC HỆ THỐNG THÔNG MINH

Assignment 07

Họ tên: Phan Văn Khôi
Mã sinh viên: B23DCCN447
Lớp: D23CTPM01-B
Giảng viên PGS. TS. Trần Đình Quế

Hà Nội – 2026

Mục Lục

Thông Tin Project **1**

1 Cơ Sở Lý Thuyết Về Knowledge Graph Và RAG **2**
1.1 Quá trình hình thành và vai trò . . . 2
1.1.1 Từ cơ sở tri thức đến RAG . . . 2
1.1.2 Vai trò trong hệ hỏi đáp trên dữ liệu bảng . . . 2
1.2 Knowledge Graph như một cơ sở tri thức . . . 3
1.2.1 Thực thể, quan hệ, bộ ba và thuộc tính . . . 3
1.2.2 Schema, định danh và truy vấn . . . 3
1.3 RAG: lập chỉ mục, truy xuất và sinh câu trả lời . . . 3
1.3.1 Giai đoạn 1: tạo tài liệu và lập chỉ mục . . . 4
1.3.2 Giai đoạn 2: truy xuất và lọc bằng chứng . . . 4
1.3.3 Giai đoạn 3: tạo câu trả lời có ngữ cảnh . . . 4
1.4 TF-IDF ký tự và độ tương đồng cosine . . . 4
1.5 Agent điều phối và phân biệt với GraphRAG . . . 5

2 Thiết Kế Hệ Thống Và Dữ Liệu **6**
2.1 Hai dataset và phạm vi hỏi đáp . . . 6
2.2 Schema graph và nguồn văn bản . . . 6
2.3 Chỉ mục, sinh câu trả lời và định tuyến . . . 7

3 Thực Nghiệm Trên Amazon Stock Price **8**
3.1 Khảo sát và phân bố dữ liệu . . . 8
3.2 Kết quả xây graph và chỉ mục RAG . . . 8
3.3 Câu hỏi kiểm chứng . . . 9

4 Thực Nghiệm Trên diabetes_large **10**
4.1 Khảo sát, nhãn và phân bố . . . 10
4.2 Kết quả xây graph và chỉ mục RAG . . . 10
4.3 Câu hỏi kiểm chứng . . . 11

5 Triển Khai Agent Trên Localhost **12**
5.1 CLI, web và API . . . 12
5.2 Giao diện Amazon: câu hỏi graph và RAG . . . 12
5.3 Giao diện diabetes_large: câu hỏi graph và RAG . . . 13

6 Đánh Giá, Giới Hạn Và Kết Luận **15**
6.1 Kiểm chứng khả năng tái lập 15
6.2 Giới hạn và hướng phát triển 15
6.3 Kết luận 15

Danh Mục Hình

3.1 EDA Amazon Stock Price xuất từ output notebook Assignment 07 8

4.1 EDA diabetes_large xuất từ output notebook Assignment 07 10

5.1 Giao diện Amazon trả lời câu hỏi Close ngày 05/12/2023 bằng Knowledge Graph 13
5.2 Giao diện Amazon trả lời câu hỏi tóm tắt bằng RAG và Qwen3 13
5.3 Giao diện diabetes trả lời tỷ lệ nhãn 1 ở nữ bằng Knowledge Graph 14
5.4 Giao diện diabetes trả lời câu hỏi tóm tắt bằng RAG và Qwen3 14

Danh Mục Bảng

1.1 Vai trò của các nhánh trong agent Assignment 07 5

2.1 Tổng quan hai CSV gốc 6
2.2 Biểu diễn KG và corpus RAG của hai bài toán 6
2.3 Cấu hình thực tế trong hai file deploy 7

3.1 Đầu ra agent Amazon đã lưu trong notebook 9

4.1 Đầu ra agent diabetes_large đã lưu trong notebook 11

Thông Tin Project

GitHub project: https://github.com/KhoiF/PTHTTM

Báo cáo này tổng hợp hai notebook và hai ứng dụng cục bộ của Assignment 07:

- **Amazon Stock Price:** `amazon_kb_rag_agent.ipynb` và `deploy.py`;
- **diabetes_large:** `diabetes_kb_rag_agent.ipynb` và `deploy.py`.

Hai notebook thực hiện khảo sát dữ liệu, xây Knowledge Graph (KG), lập chỉ mục Retrieval-Augmented Generation (RAG), rồi kiểm tra agent. Hai file deploy cung cấp giao diện dòng lệnh và web Flask trên localhost. Các số liệu, hình EDA và câu trả lời minh họa dưới đây được lấy từ output đã lưu của notebook.

Chương 1

Cơ Sở Lý Thuyết Về Knowledge Graph Và RAG

1.1 Quá trình hình thành và vai trò

1.1.1 Từ cơ sở tri thức đến RAG

Trước khi mô hình ngôn ngữ lớn được sử dụng rộng rãi, các hệ biểu diễn tri thức đã tổ chức khái niệm và sự kiện thành thực thể, quan hệ hoặc luật. Cơ sở tri thức (knowledge base, KB) là tập thông tin được lưu theo một cấu trúc cho phép truy vấn. Knowledge Graph (KG) là một cách tổ chức KB bằng đồ thị: nút biểu diễn đối tượng, cạnh biểu diễn liên hệ, còn thuộc tính ghi lại thông tin mô tả. Vì thế, KB là vai trò của kho tri thức; KG là một dạng biểu diễn cụ thể của kho đó. Bản tổng quan của Phạm Mai Linh đặt KG và RAG trong dòng phát triển từ mạng ngữ nghĩa, Semantic Web đến các hệ hỏi đáp hiện đại [1].

Trong Semantic Web, W3C chuẩn hóa mô hình RDF để phát biểu dữ kiện bằng bộ ba chủ thể–vị từ–đối tượng. Đặc tả RDF 1.1 mô tả đồ thị như một tập các bộ ba có hướng [2]. Năm 2012, Google giới thiệu Knowledge Graph cho tìm kiếm, nhấn mạnh việc nhận diện thực thể và mối quan hệ thay vì chỉ đối sánh các chuỗi ký tự [3]. Các mốc này giải thích vì sao truy vấn về Amazon, tháng 12/2023 và phiên 05/12/2023 cần được gắn với những định danh rõ ràng.

Ở hướng xử lý văn bản, DrQA năm 2017 kết hợp truy xuất tài liệu với mô hình đọc hiểu để trả lời câu hỏi mở [4]. REALM năm 2020 đưa truy xuất tri thức vào quá trình tiền huấn luyện mô hình ngôn ngữ [5]. Cũng trong năm 2020, Lewis và cộng sự đặt tên Retrieval-Augmented Generation (RAG) cho kiến trúc kết hợp bộ truy xuất với mô hình sinh câu trả lời [6]. Những công trình này là bối cảnh lịch sử; bộ truy xuất của Assignment 07 dùng TF-IDF thưa, không phải bộ mã hóa truy xuất dày đặc của công trình RAG gốc.

1.1.2 Vai trò trong hệ hỏi đáp trên dữ liệu bảng

KG phù hợp với câu hỏi có khóa rõ ràng: tìm phiên theo ngày, lần theo năm–tháng–phiên, hoặc đọc thống kê của một nhóm. RAG phù hợp với câu hỏi mô tả mà người dùng diễn đạt tự do: tài liệu liên quan được truy xuất rồi đưa cho mô hình ngôn ngữ để diễn đạt câu trả lời. Với câu hỏi cần tính trung bình, tổng hoặc tỷ lệ trên nhiều bản ghi, phép tính trực tiếp từ CSV giữ được quy tắc tính toán tường minh. Ba cách xử lý này bổ sung cho nhau nhưng không có cách nào tự bảo đảm câu trả lời đúng.

Nguồn dữ liệu cũng quyết định phạm vi kết luận. KB và corpus RAG của bài tập chỉ xuất phát từ hai CSV đã cung cấp; giá Amazon là giá lịch sử đến 05/12/2023, còn tỷ lệ diabetes là thống kê của tập quan sát. Trả kèm ID nguồn giúp kiểm tra câu trả lời, nhưng ID không thay thế bước đối chiếu nội dung và không biến tương quan thành quan hệ nhân quả.

1.2 Knowledge Graph như một cơ sở tri thức

1.2.1 Thực thể, quan hệ, bộ ba và thuộc tính

Xét đồ thị có hướng $G = (V, E)$, với V là tập nút và $E \subseteq V \times R \times V$ là tập cạnh gắn nhãn từ tập quan hệ R. Một cạnh có thể đọc thành bộ ba (s, p, o): chủ thể s liên hệ với đối tượng o qua vị từ p. Với dữ liệu Amazon, ví dụ

$$(\texttt{year:2023},\ \texttt{has_month},\ \texttt{month:2023-12}) \tag{1.1}$$

cho biết node năm nối với node tháng. Bước tiếp theo, cạnh `has_session` nối tháng với phiên `day:2023-12-05`. Trong dữ liệu diabetes, node dataset nối với node nhóm giới tính nữ qua `has_cohort`; từ node nhóm có các cạnh đến node kết quả nhãn 0 và 1. Đây là quan hệ về *cấu trúc dữ liệu thống kê*, không phải quan hệ lâm sàng giữa những người bệnh.

Thuộc tính là cặp tên–giá trị gắn với nút hoặc cạnh. Node phiên giao dịch lưu `Open`, `High`, `Low`, `Close`, `Adj_Close` và `Volume`; node nhóm lưu số bản ghi, số nhãn 1 và tỷ lệ. Chẳng hạn, node `day:2023-12-05` có `Close = 146.880005`. Ở graph diabetes, số nhãn 1 còn là thuộc tính `count` của cạnh `positive_count`. Phân biệt cạnh với thuộc tính giúp đọc đúng kết quả: cạnh nêu loại liên kết, còn thuộc tính nêu giá trị của liên kết hoặc thực thể.

1.2.2 Schema, định danh và truy vấn

Schema graph xác định các loại nút, loại cạnh và thuộc tính được phép có. Với Amazon, đường đi hợp lệ là cổ phiếu → năm → tháng → phiên. Với diabetes, đường đi là dataset → nhóm thống kê → kết quả nhãn. Định danh như `day:2023-12-05` và `cohort:gender:Female` phải nhất quán để một truy vấn trỏ đúng node. Trước khi xây graph, cần kiểm tra kiểu ngày, tên cột, giá trị thiếu và quy tắc nhóm; sau khi xây, cần đối chiếu thuộc tính với CSV nguồn.

Về lý thuyết, graph cho phép tìm node theo khóa, đi theo cạnh và nối nhiều bước quan hệ. Trong Assignment 07, `networkx.DiGraph` lưu graph có nhãn quan hệ trong thuộc tính cạnh; `query_graph` lấy trực tiếp node phiên hoặc nhóm theo định danh. Đây là graph thuộc tính cài bằng NetworkX. Nó không phải kho RDF với URI/SPARQL, không có ontology lâm sàng, và code không thực hiện suy luận nhiều bước. Việc nêu các khả năng tổng quát của KG không có nghĩa là ứng dụng đã triển khai tất cả các khả năng ấy.

Nếu ngày không có phiên hoặc nhóm không tồn tại, tra cứu trả về thiếu dữ liệu. Nếu cần trung bình trên một khoảng ngày, agent dùng trực tiếp những hàng CSV tương ứng; graph không tham gia vào phép tính khoảng ngày. Quy tắc này giữ ranh giới giữa một sự kiện đã lưu trong KB và một thống kê được tính khi có câu hỏi.

1.3 RAG: lập chỉ mục, truy xuất và sinh câu trả lời

RAG đưa thông tin từ một nguồn bên ngoài mô hình vào ngữ cảnh lúc hỏi đáp. Mô hình sinh không cần ghi nhớ từng hàng CSV trong trọng số của nó; thay vào đó, ứng dụng chuẩn bị corpus, tìm bằng chứng phù hợp, rồi yêu cầu mô hình trả lời từ bằng chứng đó [6]. Luồng tổng quát là

$$\begin{aligned} \text{CSV} &\longrightarrow \text{tài liệu có ID} \longrightarrow \text{chỉ mục}, \\ \text{câu hỏi + chỉ mục} &\longrightarrow \text{top-}k \text{ bằng chứng}, \\ \text{câu hỏi + bằng chứng} &\longrightarrow \text{LLM} \longrightarrow \text{câu trả lời + ID nguồn}. \end{aligned} \tag{1.2}$$

1.3.1 Giai đoạn 1: tạo tài liệu và lập chỉ mục

Đơn vị truy xuất phải đủ ngắn để một kết quả có ý nghĩa. Amazon tạo một tài liệu tổng quan, một tài liệu cho mỗi phiên và một tài liệu thống kê theo tháng; ví dụ `AMZN:2023-12-05` chứa các chỉ số OHLCV của phiên đó. Diabetes tạo tài liệu tổng quan `DIABETES:overall` và tài liệu cho từng nhóm, như `DIABETES:gender:Female`. Mỗi tài liệu có trường `id` và `text`. Các tài liệu này vốn ngắn và được tạo từ hàng hoặc nhóm thống kê, nên code không chia tài liệu dài thành các đoạn chồng lấn như trong một số hệ RAG dùng văn bản tự do.

Sau khi tạo corpus, `TfidfVectorizer` học từ vựng n-gram trên toàn bộ tài liệu và biến chúng thành ma trận thưa. Chỉ mục được giữ trong bộ nhớ của agent cùng danh sách tài liệu; không có cơ sở dữ liệu vector hoặc dịch vụ tìm kiếm bên ngoài. Hai bước xây tài liệu và xây chỉ mục chỉ chạy khi khởi tạo agent, trước khi tiếp nhận câu hỏi.

1.3.2 Giai đoạn 2: truy xuất và lọc bằng chứng

Câu hỏi được biến đổi bằng chính vectorizer đã học trên corpus, để vector câu hỏi và tài liệu nằm trong cùng không gian đặc trưng. Code tính cosine similarity, sắp điểm từ cao xuống thấp, xét tối đa $k = 4$ tài liệu đầu rồi chỉ giữ tài liệu có điểm ít nhất $0{,}10$. Chẳng hạn, câu hỏi tóm tắt Amazon có thể lấy tài liệu `AMZN:overview`; câu hỏi mô tả diabetes có thể lấy `DIABETES:overall`. Truy xuất chỉ cho biết mức phù hợp của văn bản theo đặc trưng đã chọn, chưa chứng minh rằng tài liệu đủ dữ kiện để trả lời.

Nếu không còn tài liệu nào sau lọc, agent trả thông báo không tìm thấy bằng chứng và không gọi mô hình sinh. Điều này khác với trường hợp có bằng chứng nhưng bằng chứng không chứa chi tiết mà câu hỏi đòi hỏi: prompt yêu cầu mô hình nói không đủ dữ liệu, song câu trả lời vẫn cần được người đọc kiểm tra. ID nguồn trong đầu ra là ID của các tài liệu được truy xuất, không phải chứng nhận tự động cho từng mệnh đề do mô hình sinh ra.

1.3.3 Giai đoạn 3: tạo câu trả lời có ngữ cảnh

Ứng dụng ghép các đoạn truy xuất theo dạng `[ID] text` vào phần ngữ cảnh của prompt, kèm câu hỏi và chỉ dẫn trả lời ngắn bằng tiếng Việt, chỉ dùng dữ kiện trong ngữ cảnh. Hai `deploy.py` dùng Qwen3-0.6B cục bộ cho bước này [7]; mô hình được nạp khi lần đầu cần sinh và được lưu trong bộ nhớ cho các lượt tiếp theo. Câu trả lời do mô hình diễn đạt, còn danh sách `sources` được chương trình lấy từ ID tài liệu đã truy xuất.

Prompt Amazon cấm suy diễn giá hiện tại hoặc dự báo; prompt diabetes cấm chẩn đoán, tư vấn điều trị và suy luận rằng một nhóm dữ liệu gây bệnh. Những ràng buộc này giảm nguy cơ trả lời vượt phạm vi, nhưng không thay thế kiểm tra nguồn. Các câu hỏi đòi phép tính chính xác được agent chuyển sang graph hoặc CSV, thay vì yêu cầu Qwen tự tính trên các đoạn văn.

1.4 TF-IDF ký tự và độ tương đồng cosine

Trong bài tập này, `TfidfVectorizer` dùng `analyzer="char_wb"` và n-gram ký tự dài 3–5. Bộ phân tích này tạo n-gram bên trong ranh giới từ, có đệm khoảng trắng; vì vậy tên cột tiếng Anh, mã nguồn và cụm tiếng Việt vẫn có thể chia sẻ một số mảnh ký tự. Đây là biểu diễn *thưa* dựa trên dạng chữ, không phải embedding ngữ nghĩa. Câu hỏi dùng từ khác xa corpus vẫn có thể bị bỏ sót.

Với N tài liệu và n-gram t, đặt $\mathrm{df}(t)$ là số tài liệu chứa t. Theo cấu hình mặc định có làm trơn của scikit-learn, trọng số trước chuẩn hóa là

$$w_{t,d} = \mathrm{tf}(t,d)\left[\log\left(\frac{N+1}{\mathrm{df}(t)+1}\right)+1\right], \tag{1.3}$$

trong đó $\mathrm{tf}(t,d)$ là số lần t xuất hiện trong tài liệu d. N-gram xuất hiện ở nhiều tài liệu có trọng số phân biệt thấp hơn n-gram hiếm. Vector tài liệu và câu hỏi được chuẩn hóa theo chuẩn L_2; điểm so khớp là

$$\mathrm{cosine}(q,d) = \frac{q \cdot d}{\|q\|_2 \|d\|_2}. \tag{1.4}$$

Với các vector đã chuẩn hóa, điểm này bằng tích vô hướng của chúng. Công thức và hành vi `TfidfVectorizer` được mô tả trong tài liệu scikit-learn [8]. Ngưỡng $0{,}10$ là quy tắc lọc thực dụng của bài tập, không phải ngưỡng bảo đảm độ đúng.

1.5 Agent điều phối và phân biệt với GraphRAG

Agent nhận câu hỏi rồi chọn công cụ theo các quy tắc có thứ tự. Amazon ưu tiên khoảng ngày để tính trực tiếp trên CSV, ngày đơn để tra node phiên, tháng hoặc năm để kiểm tra graph rồi tính thống kê CSV; câu hỏi mô tả mới vào RAG. Diabetes từ chối yêu cầu chẩn đoán, trả danh sách cột theo schema CSV, tra graph cho thống kê nhóm và tính trực tiếp cho thống kê toàn dataset; câu hỏi mô tả mới vào RAG. Nếu câu hỏi ngoài phạm vi, agent trả lời ngoài phạm vi thay vì gọi mô hình.

Bảng 1.1: Vai trò của các nhánh trong agent Assignment 07

Nhánh	**Dạng câu hỏi**	**Nguồn tạo kết quả**
Graph	Phiên theo ngày, tỷ lệ theo nhóm	Thuộc tính node/cạnh đã xây từ CSV
CSV	Khoảng ngày, thống kê toàn dataset	Phép tính trực tiếp trên các hàng CSV
RAG + LLM	Tóm tắt, mô tả dữ liệu	Tài liệu truy xuất và Qwen3 diễn đạt

Đầu ra có `answer`, `sources` và `tools`. Trường `tools` cho biết đường xử lý thực sự chạy: chẳng hạn `knowledge_graph`, `csv_statistics` hoặc cặp `rag_retrieval` và `local_llm`. Một câu hỏi theo tháng có thể dùng cả graph lẫn CSV, nhưng không đưa bộ ba graph và tài liệu RAG vào cùng một prompt.

GraphRAG là tên gọi cho những kiến trúc đưa cấu trúc đồ thị vào bước truy xuất hoặc tổng hợp ngữ cảnh cho mô hình sinh; một hướng nghiên cứu năm 2024 dùng đồ thị và tóm tắt cộng đồng để trả lời câu hỏi bao quát corpus [9]. Tài liệu tham khảo của Phạm Mai Linh mô tả một đường ống truy xuất kép từ graph và văn bản [1]. Đó là kiến trúc để đối chiếu về mặt lý thuyết. Assignment 07 hiện triển khai *agent định tuyến giữa KG, CSV và RAG*; nó không triển khai đường ống GraphRAG truy xuất kép và không chứng minh khả năng suy luận nhiều bước.

Chương 2

Thiết Kế Hệ Thống Và Dữ Liệu

2.1 Hai dataset và phạm vi hỏi đáp

Bảng 2.1: Tổng quan hai CSV gốc

Dataset	Bản ghi	Cột CSV	Thông tin chính
Amazon Stock Price	6.684	7	Date, OHLC, Adj Close, Volume; lịch sử 15/05/1997–05/12/2023
diabetes_large	100.000	9	Đặc điểm nhân khẩu, chỉ số và nhãn `diabetes` nhị phân

File Amazon được đọc theo ngày và sắp tăng dần. File diabetes có chín cột gốc; hàm nạp tạo thêm `age_group` theo bốn khoảng 0–17, 18–39, 40–59 và 60 tuổi trở lên. Vì vậy DataFrame sau xử lý có mười cột. Các khoảng tuổi chỉ dùng để mô tả dữ liệu, không phải ngưỡng y khoa.

2.2 Schema graph và nguồn văn bản

Bảng 2.2: Biểu diễn KG và corpus RAG của hai bài toán

	Amazon Stock Price	diabetes_large
Node	AMZN, năm, tháng, phiên giao dịch	Dataset, nhóm thống kê, kết quả nhãn 0/1
Quan hệ	`has_year`, `has_month`, `has_session`	`has_cohort`, `positive_count`, `negative_count`
Thuộc tính	OHLC, Adj Close, Volume của phiên	Số bản ghi, số nhãn 1, tỷ lệ của nhóm
Tài liệu RAG	Tổng quan, một tài liệu mỗi phiên, một tài liệu mỗi tháng	Tổng quan và một tài liệu mỗi nhóm

Không tạo node theo từng người bệnh trong diabetes. Điều này giữ KG ở mức thống kê, phù hợp với câu hỏi tổng hợp và tránh suy diễn rằng hai bản ghi có đặc điểm giống nhau là cùng một cá nhân. Mỗi tài liệu có ID cố định như `AMZN:2023-12-05`, `AMZN:month:2023-12` hoặc `DIABETES:gender:Female`.

2.3 Chỉ mục, sinh câu trả lời và định tuyến

Bảng 2.3: Cấu hình thực tế trong hai file deploy

Thành phần	**Cấu hình**
Graph	`networkx.DiGraph`, tạo khi khởi tạo agent
Chỉ mục	TF-IDF ký tự, n-gram 3–5; ma trận sparse trong bộ nhớ
Truy xuất	Cosine similarity; tối đa 4 tài liệu, điểm tối thiểu 0,10
Mô hình sinh	`Qwen/Qwen3-0.6B`; tối đa 180 token mới; không bật thinking mode
Deploy	Flask bind `127.0.0.1`; CLI và `POST /api/chat`

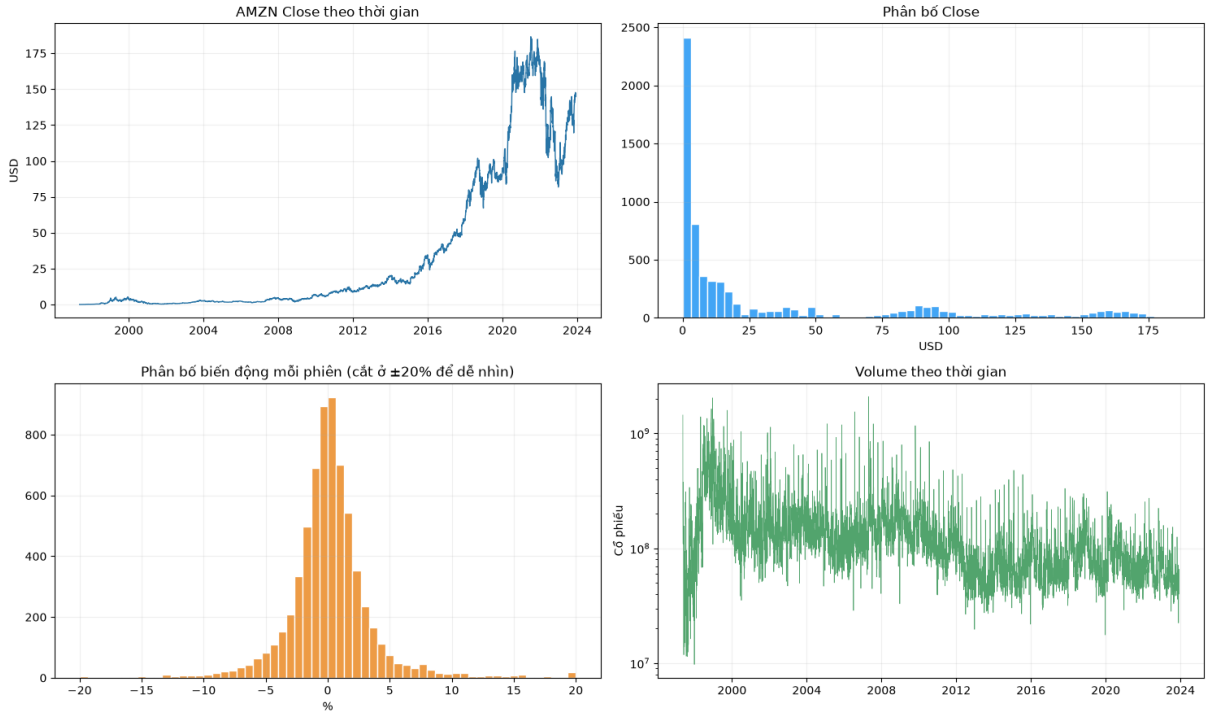
Amazon ưu tiên câu hỏi có hai ngày, một ngày, tháng, năm hoặc "mới nhất" để tra graph/CSV. diabetes ưu tiên từ chối yêu cầu chẩn đoán, liệt kê schema và trả thống kê theo nhóm hoặc toàn bộ dataset. Câu hỏi mô tả mới đi vào RAG. Prompt yêu cầu Qwen3 trả lời bằng tiếng Việt và chỉ dùng ngữ cảnh truy xuất; giao diện luôn hiển thị ID nguồn bên cạnh câu trả lời.

Chương 3

Thực Nghiệm Trên Amazon Stock Price

3.1 Khảo sát và phân bố dữ liệu

Notebook đọc 6.684 phiên từ 15/05/1997 đến 05/12/2023, không có ngày giao dịch trùng hoặc giá trị thiếu ở các cột bắt buộc. Bốn ô trong Hình 3.1 lần lượt thể hiện diễn biến Close, histogram Close, phân bố phần trăm thay đổi mỗi phiên và Volume theo thời gian. Phân bố biến động được cắt hiển thị tại $\pm 20\%$ để phần trung tâm dễ quan sát; dữ liệu gốc không bị cắt khi xây KB.



Hình 3.1: EDA Amazon Stock Price xuất từ output notebook Assignment 07

Giá theo thời gian có xu hướng và phạm vi thay đổi lớn. Vì thế, histogram gộp toàn bộ giai đoạn chỉ mô tả tập quan sát lịch sử, không phải phân phối giá cố định cho các năm tiếp theo.

3.2 Kết quả xây graph và chỉ mục RAG

KG Amazon có **7.032 node** và **7.031 cạnh**. Một đường đi mẫu nối AMZN đến nút năm 1997, nút tháng 05/1997, rồi đến phiên ngày 15/05/1997. Truy vấn node ngày 05/12/2023 cho `Close`

= 146.880005; đối chiếu với CSV trong notebook cho cùng giá trị, báo cáo làm tròn thành 146,88 USD.

Corpus có **7.005 tài liệu**; ma trận TF-IDF có kích thước 7.005×58.345. Với truy vấn về Amazon tháng 12/2023, tài liệu `AMZN:month:2023-12` được xếp đầu trong ví dụ notebook với điểm cosine $0{,}2643$. Các tài liệu lân cận có thể được trả thêm vì truy vấn mang nhiều cụm chung như "Amazon", "Close" và "Volume"; vì vậy cần đọc ID nguồn cùng nội dung.

3.3 Câu hỏi kiểm chứng

Bảng 3.1: Đầu ra agent Amazon đã lưu trong notebook

Câu hỏi	**Kết quả**	**Công cụ**
Close ngày 05/12/2023?	146,8800 USD; nguồn `AMZN:2023-12-05`	KG
Close từ 01 đến 05/12/2023?	3 phiên; trung bình 146,2500 USD; min 144,8400; max 147,0300 USD	CSV
Phiên cuối trong dữ liệu?	05/12/2023; Close 146,8800 USD	KG
Tóm tắt dữ liệu Amazon AMZN?	6.684 phiên từ 15/05/1997 đến 05/12/2023; nguồn `AMZN:overview`	RAG + LLM

Giá ở phiên cuối chỉ là giá cuối *trong CSV*; ứng dụng không truy cập giá thị trường hiện tại và không dự báo giá tương lai trong Assignment 07.

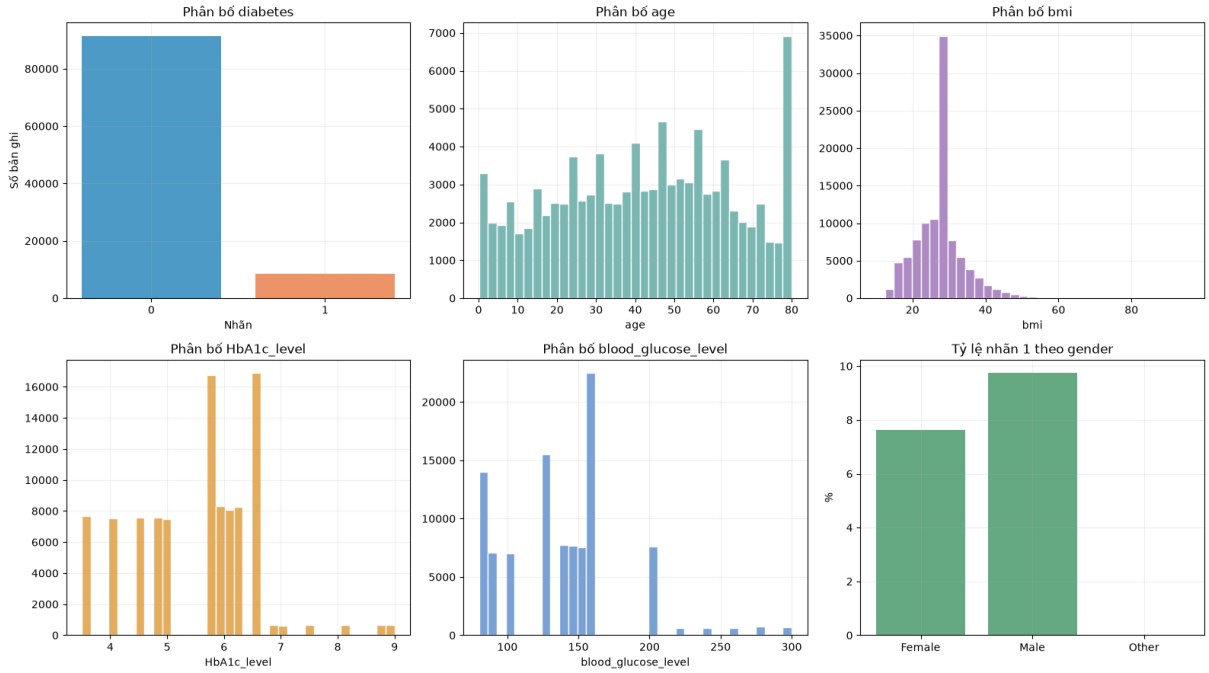
Chương 4

Thực Nghiệm Trên diabetes_large

4.1 Khảo sát, nhãn và phân bố

CSV có **100.000 bản ghi** và chín cột gốc. Notebook ghi nhận **3.854 dòng trùng toàn bộ thuộc tính**; các dòng này vẫn được giữ để thống kê phản ánh đúng tập dữ liệu cung cấp. Không có giá trị thiếu ở chín cột được dùng. Nhãn `diabetes=1` có 8.500 bản ghi, tương đương **8,50%**; nhãn 0 có 91.500 bản ghi.

Hình 4.1 cho thấy phân bố nhãn, tuổi, BMI, HbA1c, glucose và tỷ lệ nhãn 1 theo giới tính. Sự khác nhau giữa các nhóm là mô tả của dataset, không xác lập quan hệ nhân quả.



Hình 4.1: EDA diabetes_large xuất từ output notebook Assignment 07

4.2 Kết quả xây graph và chỉ mục RAG

KG gồm **20 node** và **51 cạnh**. Ví dụ node `cohort:gender:Female` nối từ node dataset, rồi nối đến hai node kết quả nhãn. Thuộc tính node cho biết 58.552 bản ghi nữ, 4.461 bản ghi nhãn 1 và tỷ lệ 7,62%. Notebook đối chiếu số lượng và số nhãn 1 với phép lọc trực tiếp từ CSV.

Corpus có **18 tài liệu**; ma trận TF-IDF có kích thước 18×1.351. Trong ví dụ truy xuất *"Tóm tắt diabetes gender Female"*, tài liệu `DIABETES:gender:Female` đứng đầu với điểm $0{,}4447$. Tài liệu nhóm Male và Other cũng xuất hiện trong top-k do có nhiều thuật ngữ chung; điểm truy xuất không được xem là tỷ lệ bệnh.

4.3 Câu hỏi kiểm chứng

Bảng 4.1: Đầu ra agent diabetes_large đã lưu trong notebook

Câu hỏi	**Kết quả**	**Công cụ**
Tỷ lệ diabetes ở nữ?	58.552 bản ghi, 4.461 nhãn 1, tỷ lệ 7,62%	KG
Tỷ lệ toàn dataset?	100.000 bản ghi, 8.500 nhãn 1, tỷ lệ 8,50%	CSV
Tỷ lệ nhóm 60+?	25.055 bản ghi, 4.932 nhãn 1, tỷ lệ 19,68%	KG
Tóm tắt diabetes_large?	100.000 bản ghi; 8.500 nhãn 1 và 91.500 nhãn 0; nguồn `DIABETES:overall`	RAG + LLM

Ứng dụng chỉ trình bày thống kê của tập quan sát. Đầu ra không dùng để chẩn đoán, ước lượng nguy cơ cá nhân hoặc quyết định điều trị.

Chương 5

Triển Khai Agent Trên Localhost

5.1 CLI, web và API

Mỗi `deploy.py` khởi tạo graph và chỉ mục khi agent được tạo. Chế độ CLI dùng `--question`; chế độ web dùng `--web`. Flask nhận `POST /api/chat` với JSON chứa trường `question` và trả về `answer`, `sources`, `tools`. Input rỗng hoặc sai kiểu được từ chối. Ứng dụng bind `127.0.0.1`; cổng mặc định là 8007 cho Amazon và 8008 cho diabetes.

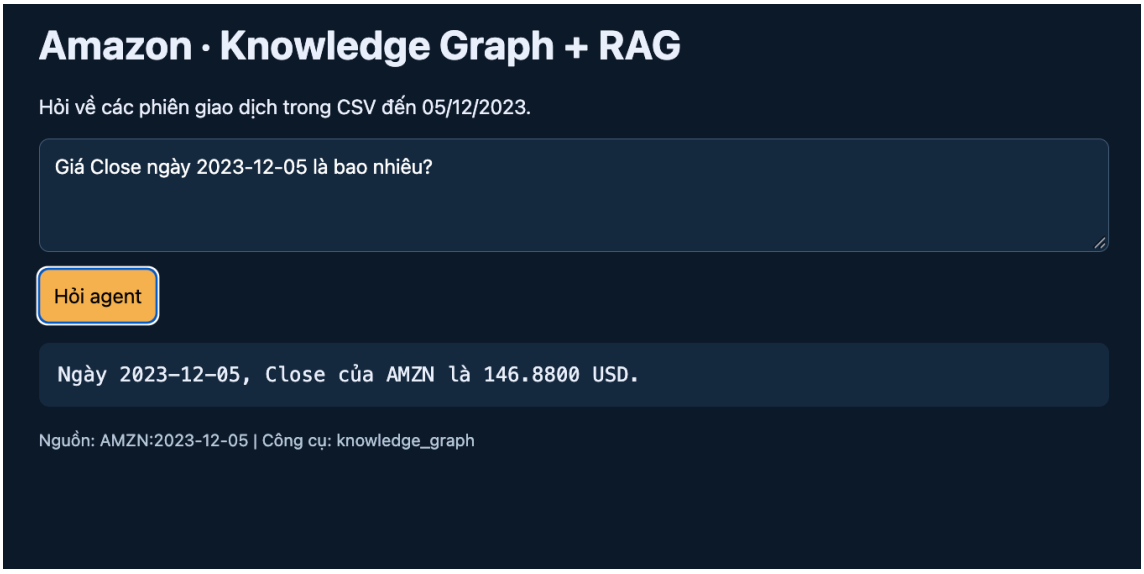
```
python "src/Assignment 07/Amazon Stock Price/deploy.py" --web --port 8007
python "src/Assignment 07/diabetes_large/deploy.py" --web --port 8008
```

Listing 5.1: Khoi dong hai giao dien web tu thu muc goc project

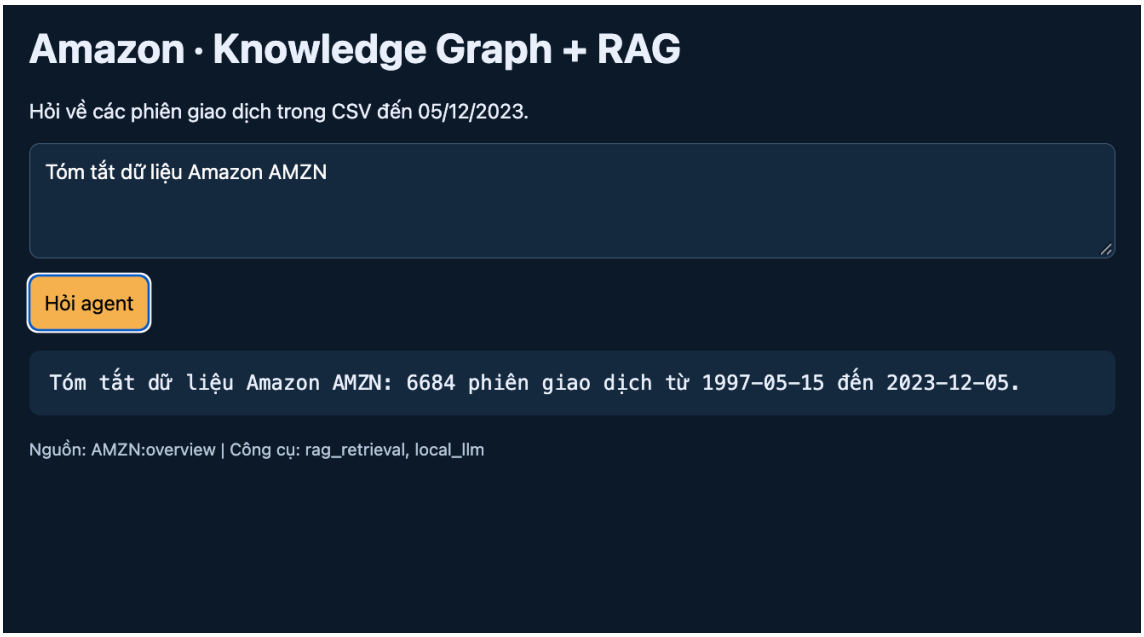
Mỗi lệnh cần chạy ở một terminal riêng. Mở `http://127.0.0.1:8007` hoặc `http://127.0.0.1:8008`, nhập câu hỏi rồi xem câu trả lời cùng nguồn và công cụ. Qwen3 được tải từ cache khi gặp câu hỏi đi vào nhánh RAG; lần tải model đầu tiên cần kết nối mạng. Các câu hỏi tra graph hoặc thống kê CSV không cần gọi LLM.

5.2 Giao diện Amazon: câu hỏi graph và RAG

Với câu hỏi về `Close` ngày 05/12/2023, agent dùng graph và hiển thị nguồn `AMZN:2023-12-05`. Câu hỏi tóm tắt dùng RAG, sau đó gọi LLM cục bộ. Đầu ra ghi hai công cụ `rag_retrieval` và `local_llm`. Nguồn là `AMZN:overview`.



Hình 5.1: Giao diện Amazon trả lời câu hỏi Close ngày 05/12/2023 bằng Knowledge Graph



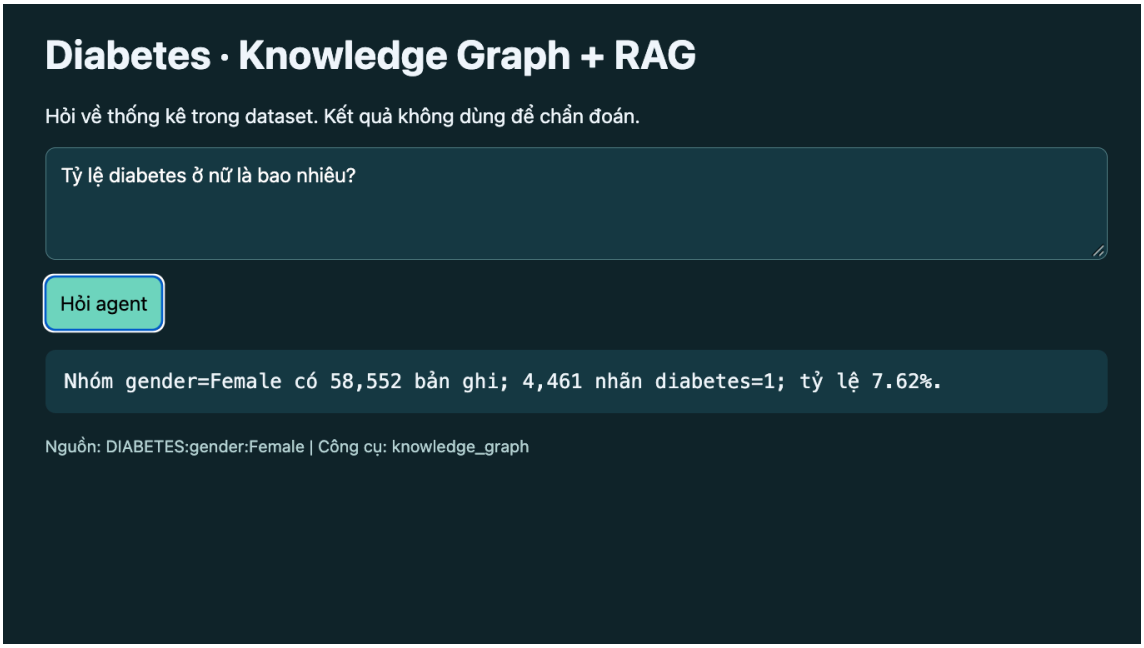
Hình 5.2: Giao diện Amazon trả lời câu hỏi tóm tắt bằng RAG và Qwen3

5.3 Giao diện diabetes_large: câu hỏi graph và RAG

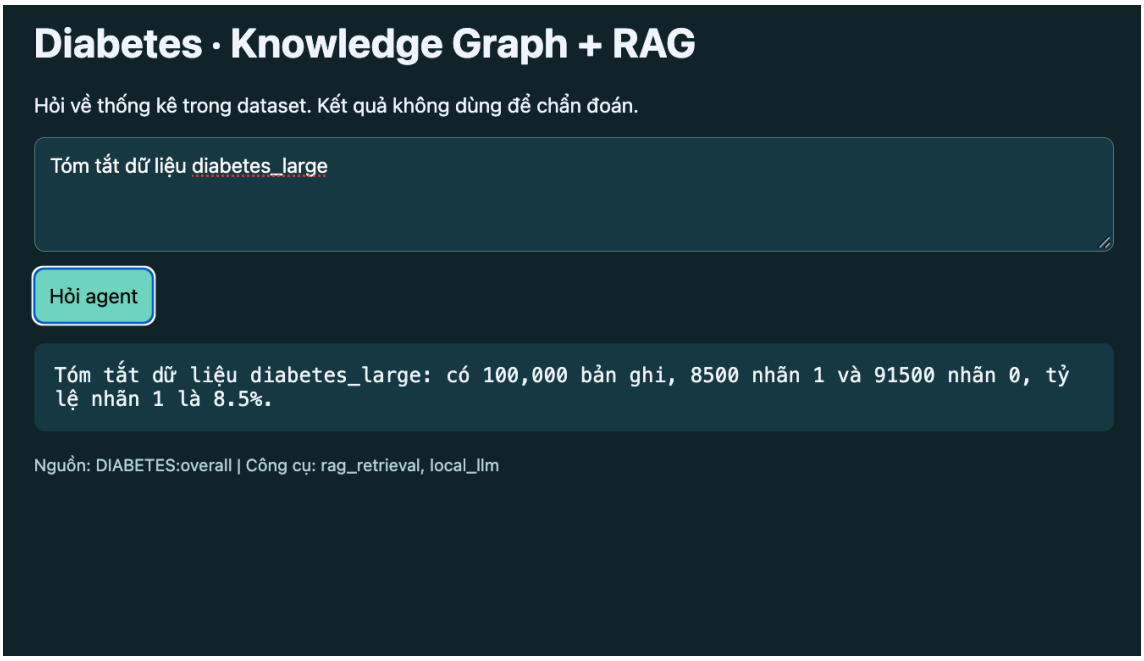
Với câu hỏi tỷ lệ nhãn 1 ở nữ, agent đọc node nhóm. ID nguồn được hiển thị là:

`DIABETES:gender:Female`

Với câu hỏi tóm tắt dataset, agent truy xuất tài liệu tổng quan `DIABETES:overall` và dùng Qwen3 để diễn đạt câu trả lời.



Hình 5.3: Giao diện diabetes trả lời tỷ lệ nhãn 1 ở nữ bằng Knowledge Graph



Hình 5.4: Giao diện diabetes trả lời câu hỏi tóm tắt bằng RAG và Qwen3

Chương 6

Đánh Giá, Giới Hạn Và Kết Luận

6.1 Kiểm chứng khả năng tái lập

Hai notebook đều có 21 cell và đã chạy hết các cell code, gồm cell `RUN_LOCAL_LLM=True`; output được lưu trong notebook, không có cell lỗi. Các câu hỏi kiểm chứng số liệu được đối chiếu với CSV. Kiểm tra API bao gồm trang web, payload hợp lệ, payload sai kiểu, ngày không có phiên, câu hỏi ngoài phạm vi và yêu cầu chẩn đoán. Trong các ví dụ RAG, đầu ra có `tools` chứa `rag_retrieval` và `local_llm`, cùng ID tài liệu nguồn.

6.2 Giới hạn và hướng phát triển

- TF-IDF ký tự dựa vào mức giống văn bản; truy vấn diễn đạt xa từ vựng corpus có thể không tìm được tài liệu phù hợp. Bài tập chưa có bộ câu hỏi gán nhãn để đo Recall@k hoặc groundedness một cách hệ thống.
- Qwen3-0.6B có thể rút gọn hoặc diễn đạt thiếu chi tiết. ID nguồn giúp kiểm tra lại, nhưng không tự chứng minh rằng mọi mệnh đề trong câu trả lời đều đúng.
- Giá Amazon dừng ở 05/12/2023. Dữ liệu diabetes là dữ liệu quan sát với nhãn mất cân bằng; tỷ lệ nhóm không phải quan hệ nhân quả hoặc công cụ chẩn đoán.
- Flask development server phục vụ demo localhost. Khi triển khai cho nhiều người dùng, cần bổ sung giới hạn request, quản lý tài nguyên model và cơ chế vận hành phù hợp.

6.3 Kết luận

Assignment 07 xây hai hệ hỏi đáp cục bộ trên dữ liệu bảng bằng cách kết hợp Knowledge Graph, thống kê CSV và RAG. Graph xử lý câu hỏi có khóa rõ ràng, phép tính trên CSV giữ độ chính xác cho câu hỏi số, còn RAG cung cấp ngữ cảnh có nguồn cho câu hỏi mô tả. Hai notebook và hai ứng dụng CLI/web cho thấy luồng từ khảo sát dữ liệu, biểu diễn tri thức, truy xuất đến trả lời bằng agent. Việc giữ nguồn và nêu giới hạn của từng dataset là điều kiện cần để sử dụng kết quả đúng mục đích.

Tài Liệu Tham Khảo

[1] Phạm Mai Linh, *Chương 04: Knowledge Graph và RAG*, Tiểu luận học phần Phát triển các hệ thống thông minh, Học viện Công nghệ Bưu chính Viễn thông, 2026.

[2] W3C, *RDF 1.1 Concepts and Abstract Syntax*, 2014. https://www.w3.org/TR/rdf-concepts/.

[3] A. Singhal, Google, *Introducing the Knowledge Graph: things, not strings*, 2012. https://blog.google/products-and-platforms/products/search/introducing-knowledge-graph-things-not/.

[4] D. Chen, A. Fisch, J. Weston, and A. Bordes, *Reading Wikipedia to Answer Open-Domain Questions*, 2017. https://arxiv.org/abs/1704.00051.

[5] K. Guu et al., *REALM: Retrieval-Augmented Language Model Pre-Training*, 2020. https://arxiv.org/abs/2002.08909.

[6] P. Lewis et al., *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*, 2020. https://arxiv.org/abs/2005.11401.

[7] Qwen, *Qwen3-0.6B Model Card*. https://huggingface.co/Qwen/Qwen3-0.6B.

[8] scikit-learn, *TfidfVectorizer Documentation*. https://scikit-learn.org/stable/modules/generated/sklearn.feature_extraction.text.TfidfVectorizer.html.

[9] D. Edge et al., *From Local to Global: A Graph RAG Approach to Query-Focused Summarization*, 2024. https://arxiv.org/abs/2404.16130.